Yulang 型推論の統合設計と証明計画

Simple-sub と Yulang2 Oracle を接続する実装手順

2026 年 10 月 21 日

1 結論と最初に直す箇所

Yulang の `research/simple-sub-intrusion` は、共通の推論核を既に持っている。Value と Effect の制約は同じ `InferenceSession` と worklist に入り、極性付き extrusion、Value 参照グラフの SCC による intrusion、一般化時のグラフ捕捉、使用ごとの freshening が存在する。したがって、これらを捨てて別の型推論器を最初から作る必要はない。[R02][R03]

統合すべき対象は、**ソースから作った制約が、伝播、コピー、一般化、使用、実行まで同じ意味を保持する仕組み**である。現在はこの接続が、構文ごとの slot、物理的な bound owner、観測用の receipt、別々の実行経路に依存している。局所補題が増えても、ソース全体についての定理の前提を閉じにくい構造になっている。

この統合方針自体は、10 月 7 日の proof architecture にも既に整理されている。そこでは生成時に証拠を導入して保持する方針が示されているが、現行の Capture などがその契約を満たしたことにはなっていない。本書の追加点は、その方針を現在の具体的なデータ所有者、閉包計算、保存命題、移行手順へ落とすことである。[R27]

最優先の具体的な修正箇所は、一般化時の `Capture` である。現在の実装では、source completion の記録をメモリ確保失敗時に省略して推論を継続する一方、その記録が残った特定の Lambda slot の辺だけを、Capture の逆向き探索に採用している。意味に必要な制約の列挙が、欠けてもよいはずの観測記録に依存している。[R04][R05]

ここに対して、本書は次の統合方針を提案する。

1. **一つの意味的な制約の記録を正本にする。** 生成元、型付きの端点、scope、必要な context を採用時に保持する。伝播用の片側 adjacency と、Capture 用の両端索引は、この同じ記録を参照する。

2. **一般化の対象を依存関係の閉包として捕捉する。** 型の表面から一回だけ逆向きに辿る方式を、意味に必要な制約についての有限の閉包計算に置き換える。外側の共有変数は同じ identity のまま残す。
3. **一使用につき一つの整合した写像族でコピーする。** 型、effect、context、permission、元の制約との対応を一緒に写す。別々の使用は local identity を共有しない。
4. **Yulang2 を操作単位の Oracle にする。** 受理だけでなく、型、handler 後の effect 残余、独立使用、実行時の handler 選択を比較する。既知の正常例を証明の都合で拒否しない。
5. **局所補題を同じ合成定理に接続する。** 制約生成、solver、一般化、型付き実行の四段階で前提と結論を揃え、別の定理にすり替えずに健全性・主型性・停止性を閉じる。

第 6 章では、Capture の閉包と freshening について、仮定を明示した一般命題を実際に証明する。第 8 章では、それを Yulang 全体の型健全性へ接続するための定理と残る証明義務を示す。現行コンパイラ全体の健全性が既に証明された、という主張ではない。

1.1 参照する版

対象	固定した識別子
監査する研究ブランチ	`research/simple-sub-intrusion`
監査 commit	`5596b1598388f629f95a44ef203e4f760b615803`
Yulang2 Oracle の既存基準	`a58eefc31e22141574b6f20c6a5748151c6d79f1`
比較した main	`d4c8bc8fc8a49173180932cd25921a2c41057354`
Simple-sub の参照実装	`9bae772624c23b52a93c1b226157e16898b4d9db`

Yulang2 の基準と main の間では、今回比較した `crates/infer`、`crates/types`、`crates/mono`、`crates/specialize` に差分がない。これは当該ディレクトリについての確認であり、リポジトリ全体の同一性を意味しない。以下のソース行番号は固定 commit に対するものとする。[R01][R16]

1.2 読み方

今すぐ実装作業を割り当てるなら第 9 章と第 10 章、設計判断を確認するなら第 3 章から第 5 章、証明を引き継ぐなら第 6 章から第 8 章を読む。第 11 章は既存の証明課題との対応、第 12 章は確認結果と文献である。提案する新しい内部型名や関数名は疑似コードであり、既存 API と混同しない。

2 現状で接続しているものと未接続のもの

2.1 共通核は存在する

候補実装の基本経路は、`parse_file`、`lower_module_with_local_source`、`CandidateInference::solve`、`ConstraintBatch`、`InferenceSession` である。source action schedule が通常の型付き worklist に制約を渡す。Value と Effect の処理を別々の推論器へ分裂させる必要はない。[R02][R03][R07]

共通操作	現在の主な所有箇所
ソース由来の制約と実行順序	`candidate_source.rs`
セッションと上下界の伝播	`yu-solver/src/lib.rs`
極性付き extrusion	`candidate_extrusion.rs`
実際の Value 参照と SCC	`candidate_value_graph.rs`
親子コピーの intrusion	`candidate_intrusion.rs`
context と effect の関係	`candidate_context.rs` と `candidate_effect.rs`
Capture と FreshUse	`candidate_scheme.rs`

ここで「一つの核」とは、全コードを一ファイルへ集めることではない。同じ意味の事実に正本が一つあり、各モジュールが同じ規則と変換を共有することである。

2.2 四つの接続が分かれている

旧 solver と候補 solver。 候補モジュールは `shadow-apply-candidate` feature で有効になり、通常経路とは別の collection mode を使う。両者が同じセッション型に入っていても、同じ意味の推論経路とは限らない。旧実装の両側 adjacency と、候補実装の片側所有にも違いがある。[R03]

LocalSource と旧 HIR。 `CandidateInference` は LocalSource の有無で preflight を分ける。移行中の橋渡しとしては理解できるが、同じ構文の制約生成規則を二か所で維持すると、それぞれに source correspondence が必要になる。[R02][R06]

内部グラフと公開主型。 候補経路の終了時には `closed_types.is_none()` と `schemes.is_empty()` が確認される。内部グラフを得たことと、一般化された公開主型が完成したことは別である。主型の比較がまだできない箇所は、Oracle 比較でも「未取得」とする必要がある。[R03]

推論と runtime evidence。 通常の playground 実行は、推論後に `RuntimeEvidence::default()` を渡す。一方、限定された operation v1 の経路は、同一 HIR に対応する receipt を runtime へ渡す。後者の成功を、一般の Call と handler の runtime bridge の完成として扱えない。[R07]

2.3 未実装と型エラーを分ける

機能	固定 commit での状態	必要な次の接続
Tuple	LocalSource に順序付き表現があるが、候補推論と runtime は明示的に未対応	product 型と全変換、左から右への評価
Method と Role	parser occurrence の inventory はある	意味的な選択義務と、選択証拠を伴う Call
Existential	generic Catch arm は rigid opening 待ちで単相に限定	fresh level による rigid 導入と非脱出
Optional record	candidate の一般的な constructor 接続が未完成	局所 adaptation と実行証拠
Level	Value と Effect の変数に実装済み	コピーと rigid 制約へ同じ不変条件を適用
反変の具体 effect 行	R252 で開いた行へ修正済み	permission の実消費と残余の公開

Tuple を Unit や group に読み替えたり、標準ライブラリの演算子をサンプル専用に直結したりして「元のサンプルが動いた」と判定してはならない。元のソースを保ち、parse、import、型推論、実行のどの段階で止まったかを記録する。[R06][R08][R09][R10]

2.4 既存の証明状態を読み違えない

`shadow_apply.rs` は、構造的な solve 成功後も残る十九種の未解決前提を列挙する。Apply の型付け、完全な呼出し意味論、provider の適合、source admission、scope 輸送、一般化と fresh use、event の出力対応などである。これは未完了を明示した設計であり、成功したテストの数で上書きできない。[R02]

`successor-proof-obligations.md` の CLOSED と CONDITIONAL-CLOSED も区別する。例えば PRINCIPAL の合成には SOURCE_ADEQUACY、ALL_VIEW、PROJECTION の前提が残る。条件付きの合成定理の証明は価値があるが、その前提が現在の source constructor から成立することを示して初めて、対象言語の主型性へ使える。[R11]

3 守るべき言語方針

3.1 Simple-sub の操作を背骨にする

基本操作は、制約生成、蓄積、伝播、level に応じた extrusion、実際の Value 参照に基づく intrusion、一般化、使用ごとの freshening とする。型候補の列挙、試行錯誤による探索、失敗した型付けを rollback して別の型付けを試す方式を主機構にしない。[R12]

異常時に private attempt を捨てることは、型付け探索のための rollback とは異なる。必要なのは、矛盾した結果や途中の scheme を公開しないことである。全内部状態・確保容量・診断順序まで元どおりにすることを、新たな健全性要件として追加しない。

3.2 SCC は値の参照関係から作る

intrusion に使う SCC は、Name がどの Value を参照するかというソース由来のグラフから作る。型の上下界のグラフの SCC で代用しない。extrusion が実際に作った親と子の型変数対に、対応する

元の Value と受け手の Value を保持する。対象の二つの Value が同じ SCC にあるときだけ、その occurrence の親子コピーを同一化する。[R12][R13]

SCC に属するという事実だけから、無関係な型変数、異なる使用の hygiene identity、正負のコピーをすべて同一化してよいわけではない。何を同一化するかは、実際の extrusion occurrence が与える。

3.3 型変数の伝播と具体型の適合を区別する

変数間の上下界は蓄積して伝播する。一方、具体型同士の optional record や method の適合は、当該の使用位置での adaptation として扱う。具体型の関係そのものに推移律を追加しない。

本書で用いる制約の充足関係は、この区別を含む。第 6 章の Capture 定理は任意の原子的な制約に対して成立するため、具体型 adaptation の推移性を仮定しない。伝播規則の正しさは、各規則が採用された言語意味論のもとで帰結を保つこととして個別に証明する。通常の部分型関係の記号を全機能に一律に使い、そこへ推移律を持ち込まない。[R12][R14]

例えば承認済みの optional record の観測では、`{foo?: string}` から `{}`、`{}` から `{foo?: int}` への局所比較は成功し得る一方、`{foo?: string}` から `{foo?: int}` への比較は成功しない。二つの具体比較の成功を、第三の成功の根拠にしてはならない。[R28]

有限な ground 断片についての補題。 有限な変数集合 W、ground な正負の端点集合 P, N、任意の局所互換関係 $K \subseteq P \times N$ を取る。変数 v の初期 lower/upper 集合を $L_0(v), U_0(v)$ とする。変数の flow $v \to w$ は、$L(v) \subseteq L(w)$ と $U(w) \subseteq U(v)$ を要求する。集合割当の整合性を $L(v) \times U(v) \subseteq K$ と定める。この有限モデルについて

$$L^*(v) = \bigcup_{u \to^* v} L_0(u), \quad U^*(v) = \bigcup_{v \to^* w} U_0(w)$$

は初期集合を包含する最小の flow 閉包である。$L_0(v) \subseteq L(v)$、$U_0(v) \subseteq U(v)$ と flow の条件を満たす整合した集合割当が存在することと、この閉包自体が整合することは同値である。

証明。 長さゼロの路から初期集合の包含を得る。路を一辺延ばすことにより flow の包含条件を得る。初期集合を包含し、flow を満たす任意の割当は、路長の帰納法により L^*, U^* を含む。したがってそ

のような整合した割当が存在すれば、その部分集合である閉包も整合する。逆に閉包が整合すれば、それ自身が求める割当である。□

ここでは K を推移的に合成していない。証明したのは有限な bound 集合のモデルであって、任意の Yulang の型や値が実現できることではない。source から replay を生成する際には、各正負の端点対を比較してよいことを、元の source rule と同じ context から正当化する必要がある。過去の具体比較の成功だけから新しい比較を追加してよいとは述べていない。

3.4 Level と hygiene identity を混同しない

level は型変数の一般化可能性と scope を管理する。構造型自体に新しい独立した level を追加しない。必要なら子に現れる変数の level から派生量を計算する。

existential の内部型は fresh level で rigid に導入する。承認済みの規律は、level l で導入した existential と level $\leq l$ の型との比較・同一化を、生成・比較時に拒むというものである。alias や bound replay が作る派生比較にも同じ guard を再適用する。構造型の判定は通常の分解と変数側の level 処理へ還元する方針であり、すべての比較経路での厳密な実装対応が必要になる。effect の handler identity、型変数 identity、実行時に作られる fresh identity は別の役割を持つ。コピー時には、それぞれの sort に対応した写像を同時に適用する。[R14]

3.5 Oracle に合わせる対象を明確にする

Yulang2 は、受理・拒否、型、effect、実行結果についての挙動の基準である。実装内部のデータ構造をすべて保存する必要はない。同時に、旧実装が受理したという事実だけで健全性は得られない。

Oracle との差異を修正するときは、実装不足、候補実装の回帰、立証済みの旧実装の不具合、明示された仕様訂正、未決着の五種類に分ける。旧実装の不具合と呼ぶには、同じソース、同じ使用、同じ実行観測を結ぶ反例か、対象の型規則を否定する具体的な証明が必要である。

4 統合後の内部契約

4.1 三つの情報を分けて一緒に保持する

一つの制約について、次の情報を持つ。

情報	意味	更新方針
元の identity	ソースの occurrence と、採用された制約の identity	不変に保つ
現在の端点	union や intrusion 後の代表に解決した端点	正規化して参照する
意味的な付帯情報	scope、context、四ポートの相関、permission、必要な導出元	同じ変換で輸送する

物理的な格納先が lower 側であっても、意味的な制約は両端と必要な payload に依存する。Capture に必要なのはこの依存関係であり、「どちらの配列に入ったか」ではない。

既存の `BoundAdmissionId`、`BoundEmissionId`、relation、typed occurrence を優先して利用する。これらで表せる情報に新しい同格の正本を追加しない。新しい索引を置く場合も、正本から導出される索引と位置付ける。

4.2 推奨する最小限の操作境界

次は既存関数を整理するための疑似コードである。

```
admit(rule, occurrence, endpoints, payload)
    -> ConstraintId

enqueue(ConstraintId, transformation)
    -> work item

transport(map, constraint)
    -> transformed endpoints and full payload

capture(roots, boundary)
    -> dependency-closed graph with shared anchors

freshen(captured_graph, use)
    -> one coherent family of fresh maps
```

`admit` は、意味的に必要な端点索引を含めて成功を確定する。その後に任意で作る trace や説明文が失敗しても、制約の意味は変わらない。索引が意味に必要であるなら、その確保失敗は private solve の失敗であり、成功結果に見せかけない。

`transport` は型だけを写す関数ではない。source identity は導出元として保持し、コピーされる local identity と、固定する outer identity を区別する。元の occurrence を新しい occurrence に偽装しない。

4.3 関数は四ポートを一まとまりとして扱う

Yulang の関数に現れる argument value、argument effect、result effect、result value を、それぞれ AV、AE、RE、RV と書く。構造的な関数比較では AV と AE が反変、RE と RV が共変になる。だが実際の Call では、四つを独立に適合させるだけでは不十分である。

同じ実引数、同じ provider、同じ元の Call、同じ use、同じ scope のもとで、四ポートが同時に実現される必要がある。ポートごとに別の provider や別の使用の証拠を選べば、個々の比較が成立しても一つの呼出しを説明できない。[R17][R18]

したがって、Call の導出が生成する情報は一つの tuple として運ぶ。四ポート、context、permission、残余への対応は、その tuple の射影とする。既存の whole-tuple transport と map の設計を、この共通契約へ接続する。

4.4 一般化と公開を別々の推論器にしない

一般化では、現在の graph の一部を、その scope と shared anchor を明記した状態で保持する。公開する型の簡約はその後段で行い、簡約の正しさは graph の意味と公開観測の一致として証明する。

これは新たな専用の quantified solver や F5 の closed scheme を中核へ戻す提案ではない。本書の「制約付き graph」という数式表現は、既存の `candidate_scheme::Graph` と ordinary Simple-sub の制約を説明するための意味論上の記法である。

4.5 成功結果の意味を段階で固定する

研究中の構造的 solve、source typing と対応が付いた結果、runtime に必要な証拠まで揃った結果を、API 上で区別する。未完の機能を拒否仕様へ変えるのではなく、「何を確認した結果か」を区別する。

production へ進める際には、同じ型付き結果を evaluator と公開型生成が消費する。独立した小さい検証器を使う場合も、同じ宣言的型規則の導出を検査するものとする。検証器を新しい推論器にしたり、solver が作った成功フラグを再確認するだけのものにしたりしない。

5 Yulang2 から再利用する操作

5.1 対応表

操作	Yulang2 の所有者	現行実装での接続
上下界追加と交差 replay	`machine/bounds.rs` の `add_lower_bound` と `add_upper_bound`	`candidate_replay_bound` と Value/Effect の apply
関数の四ポート比較	`machine/propagate.rs`	paired function と port operation
ソースの application	`lowering/expr/tail.rs` の `make_app_with_origins`	`candidate_call.rs` と source Call
注釈の極性と effect 行	`annotation/constraints.rs`	`candidate_effect.rs` の paired annotation
context の順序付き合成	`ConstraintWeights`	`ContextExpr` と `evaluate_context`
handler 残余の制約	`row_effect.rs`	source view、context relation、residual
量化と一使用内の共有	`instantiate_scheme_parts`	`prepare_candidate_fresh_use` と restore

各行の完了条件は、旧関数を見つけたことではない。新しいソース規則が作る制約、伝播先、コピー後の消費点、実行結果が同じ規則で説明できることである。詳細な位置は第 12 章の [R16] から [R21] と [R29] を参照する。

5.2 そのままの移植ではない extrusion

Yulang2 の `extrude_pos` と `extrude_neg` は元の ID を返し、`extrude_type_var` が既存の変数の level を下げ、関連する bounds を辿る。現行の極性付き親子コピーと Value-SCC intrusion は、この操作とは異なる。[R16]

よって、「Yulang2 の extrusion が動いていたから、新 extrusion と intrusion も正しい」という証明はできない。旧実装から受理例、共有関係、一般化境界の挙動を持ち込み、新実装については次を示す必要がある。

1. extrusion が作る正負のコピーの関係と、元の制約の関係が、対象境界で正しく対応する。
2. intrusion の条件を満たす occurrence では、親子を同一化しても必要な解を失わず、不正な解を追加しない。
3. コピー前後の Value 参照、context、effect、後続 work が、同じ対応を維持する。

正負のコピーを無条件に同一視することもできない。正負の情報を別に保持する Simple-sub の extrusion と、新たな同一化を正当化する Yulang の条件を分けて証明する。

5.3 R252 を一つの統合例として完成させる

R252 の修正では、反変位置の `[io]` を開いた行 `[io,'e]` として扱い、`[state]` callback の一律拒否を取り除いた。これは維持すべき修正である。再び closed `Allowance` を導入して証明を簡単にしてはならない。[R10]

ただし現在の `FormalPermission` は局所的な permission の記録であり、実際の arrival、provider、contribution、consumer の証拠ではない。受理側の CLI 回帰も、成功終了と診断なしを確認するだけで、残余 effect を確認していない。[R10]

次の実装単位は、次の一連の挙動を同時に閉じることである。

- `[io]` を要求する反変の formal へ `[state]` callback を渡せる。
- callback を実際に呼ぶと、その `[state]` は対応する結果 effect に現れる。
- handler の permission は、それが本当に扱う contribution にだけ使われる。

- 別 family、別由来の同 family、handler の外側の contribution は、規則に従って残る。
- 一般化された関数を二回使っても、局所 permission の identity が交差しない。

unused callback の受理と called callback の残余は別の観測である。片方だけを測って effect hygiene 全体の完成としない。

5.4 Oracle の不具合と未決着を分ける

`oracle-direct-provider-hygiene-counterexample-proof.md` は、その名前に反して、反例がまだ成立していないと明記する。provider の型と direct use の型の見かけの違いだけでは、同じ実行時の provider と attachment が対応しているとは限らない。[R22]

また、過去の CBV mini-language の I-Let 規則に対する反例は、その記述された定理の反例である。実際の Oracle compiler の実行バグと同じ主張にはならない。既存証明の誤りを正すことと、正常な Oracle の挙動を保存することを両立させる。

6 Capture と FreshUse の一般保存命題

この章は提案する内部契約についての数学的な証明である。抽象的な仮定をコードへ接続する場所を各節の末尾に示す。型の健全性全体を仮定して結論を言い換えるものではなく、現在の incoming-owner と freshening の問題に直接使う。

6.1 意味的な依存関係

有限個の座標の集合を V とする。座標は型変数だけでなく、effect 変数、コピー対象の scope identity、constraint payload が意味的に参照するその他の identity を含む。各座標には sort があり、適切な領域の値を割り当てる。座標間の kind・scope・level による許容条件も制約として記録するか、固定した環境の条件として明記する。割当同士を結合できなくする未記録の大域条件を置かない。

有限な制約集合を C とし、各 $c \in C$ に有限な依存集合 $dep(c) \subseteq V$ を指定する。割当が集合 D 上で一致することを $\rho \equiv_D \rho'$ と書き、次の局所性を仮定する。

$$\rho \equiv_{dep(c)} \rho' \Rightarrow (\rho \vDash c \Leftrightarrow \rho' \vDash c).$$

つまり、制約の真偽を決める情報はすべて依存集合に入っている。見えない global registry、別 use の provider、非公開の mutable context を充足判定が参照するなら、それを環境の固定パラメータとして明記するか、依存集合に含める必要がある。

一般化可能な局所座標を L、固定する外側の座標を R とし、$V = L \sqcup R$ とする。出力 root の観測が依存する局所座標の集合を $Q_0 \subseteq L$ とする。root の型だけでなく、公開する effect と必要な実行証拠の依存先もここへ含める。

6.2 補題 1 依存閉包の存在と完全性

次の列を定める。

$$Q_{n+1} = Q_n \cup \bigcup_{\substack{c \in C \\ dep(c) \cap Q_n \neq \varnothing}} (dep(c) \cap L).$$

有限性により、ある n で $Q_{n+1} = Q_n$ となる。その固定点を Q とする。捕捉する制約は

$$C_{cap} = \{ c \in C : dep(c) \cap Q \neq \varnothing \}$$

である。必要な外側の座標は同じ identity の anchor として記録する。root 自体が要求する、局所座標を持たない証拠や外側の条件も保持するが、外側の座標を経由して無関係な制約全体をコピーする必要はない。

主張。 Q は、Q_0 を含み、局所座標を共有する制約の全依存先で閉じた最小の集合である。また、捕捉しない制約は Q の座標に依存しない。

証明。 列は単調増大で L に含まれる。各真の増大は少なくとも一つの局所座標を追加するため、増大の回数は L の要素数を超えない。固定点では、Q と交わる依存集合の局所部分はすべて Q に含まれる。Q_0 を含む任意の閉じた集合は、帰納的に各 Q_n を含むため、Q も含む。最後の主張は C_{cap} の定義そのものである。□

両端と payload の逆索引があれば、一度訪れた座標と制約を記録する worklist で計算できる。計算量は捕捉する座標数と、その incidence 総数に比例する。これは型候補の探索ではなく、既存の有限グラフの到達計算である。

コードへの接続。 `shape_effect_rows` は初期集合の一部には使えるが、閉包全体にはならない。slot 4 の有無、bound の片側所有、任意欠落する receipt に依存しない、全意味的 incidence の索引が必要である。[R04][R05]

6.3 補題 2 境界を固定した解集合の分解

X を L のうち Q に入らない座標の集合、C_{rest} を C のうち捕捉しない制約の集合とする。補題 1 より、捕捉した制約は $Q \cup R$ に、残りの制約は $X \cup R$ にだけ依存する。

外側の割当 r と、C_{rest} を満たす X 上の割当 x を固定する。このとき、任意の Q 上の割当 q について

$$(q, x, r) \vDash C \Leftrightarrow (q, r) \vDash C_{cap}.$$

証明。 左辺から右辺は制約の部分集合への制限である。右辺から左辺について、C_{cap} は仮定により成立し、C_{rest} は固定した (x, r) により成立する。両集合の和が C であり、どちらも他方の局所変数に依存しないので、二つの割当を結合できる。□

特に、残りの制約を満たす x が少なくとも一つ存在する外側割当 r について、root の可能な観測は、捕捉した graph と同じ anchor から得られる観測に一致する。

この条件は省略できない。 どんな外側割当でも残りの制約が充足可能である、と主張しているのではない。既存の外側セッションが満たす条件を保ったまま、局所変数を切り出す定理である。外側の矛盾を Capture によって消すことはできない。

また、全局所座標をつなぐ制約があれば、捕捉範囲が大きくなることはある。意味的に不要な辺を省きたい場合は、別途、その省略が同じ root の観測を保存する証明を与える。単に表面から見えないという理由では省かない。

6.4 補題 3 同一の写像による FreshUse 保存

μ を Q から fresh な同じ sort の座標集合 Q' への全単射とし、R 上では恒等写像とする。実装上は局所座標への単射を新しい像集合への全単射と見ればよい。identity を含む意味的な値を扱う場合は、この写像を合法な名前変更へ拡張し、その値への作用も $\mu \cdot v$ と定義する。固定する anchor の値 $r(b)$

はその作用で変化しないことを仮定する。これは、一般化する局所名を外側が自由に参照していないという境界条件である。

各制約と root の意味が、この名前の変更について同変であると仮定する。すなわち、payload 中のすべての該当 identity を同時に写し、binder を捕獲しないときに

$$(q, r) \vDash c \Leftrightarrow (\mu_* q, r) \vDash \mu(c)$$

となる。ここで $\mu_* q$ は $\mu(a)$ に $\mu \cdot q(a)$ を割り当てる写像である。意味的な値が局所 identity を含まない通常のモデルでは、この作用は恒等であり、単に $q(a)$ を移せばよい。anchor の値が局所名を含む場合には同じ r を固定する定理は適用できず、何を shared に戻すか、あるいは外側もどう輸送するかを先に決める。

主張。 Capture 後の局所解と、FreshUse 後の局所解は一対一に対応する。root の型・effect・同じ use の証拠について、$Obs(\mu C, \mu_* \rho) = \mu \cdot Obs(C, \rho)$ という名前変更に沿った観測保存が成り立つ。外部観測が名前の α 同値で識別される場合、その観測は同じになる。

証明。 制約ごとの同変性をすべての $c \in C_{cap}$ に適用する。連言を取れば graph 全体の充足が保存される。逆写像 μ^{-1} を使えば逆方向も得られる。root と付帯証拠にも同じ変換を適用するので、観測は同じ写像に沿って対応する。□

一つの旧 identity が、一つの使用内で異なる新 identity へ写ってはならない。型変数だけを fresh にして effect context を使い回すことも、この仮定を満たさない。

コードへの接続。 `canonical_rows`、`view_remap`、`context_remap` などは、独立した都合のよい置換ではなく、一つの型付き写像族の成分でなければならない。raw ID の順序・値を意味が参照する場合は、単なる名前変更が意味を保つことを別途確認する。[R19]

6.5 系 独立使用と共有 anchor

同じ graph の二つの使用について、$\mu_1(Q)$ と $\mu_2(Q)$ を互いに素とし、どちらも R を固定する。各使用の局所解は、同じ外側割当 r のもとで独立に選べる。二つの解を合併しても名前衝突がなく、それぞれの制約を満たす。

ここで独立なのは、一般化してよい局所座標だけである。共有 store、外側の変数、nongeneric な情報を無理に独立にしてはならない。CBV の副作用を持つ値の一般化条件や、再帰 SCC 内の live root の扱いは、何を Q に入れてよいかを決める別の証明義務である。

この系は、二回使用で hygiene identity が混ざらないことを示す一方、effectful value を無条件に多相化してよいとは述べていない。

6.6 公開後の更新に必要な条件

補題 1 から 3 は、固定した graph の状態に対する定理である。Capture 後に元の局所座標へ新しい意味的な制約が入る場合、その制約が既存のコピーへ自動的に反映されるとは限らない。

公開境界では、以後の意味的な更新が shared anchor または新しい instance にだけ入ることを証明するか、影響する capture と既存 use を無効化し、再捕捉・replay する対応を持つ必要がある。再利用可能な一般化境界であることは、SCC と source schedule の所有者が示す。証明のために境界を勝手に早めたり、正常な遅延制約を拒否したりしない。

この条件を含めて初めて、snapshot の保存を推論全体の lifecycle の保存へ持ち上げられる。共有境界の型が後から変わる例、generic graph の公開後に新しい lower が届く例を、段階 1 の検査へ含める。

6.7 補題 4 ソース由来の情報を失わない構成

すべての意味的な制約レコードに、元の source occurrence か、既存レコードと変換を参照する導出記録を持たせる。次の生成経路を網羅するとする。

- source rule は、その規則の occurrence と rule tag を持つレコードを生成する。
- replay と decomposition は、親のレコードと実際に適用した演算を記録する。
- extrusion と freshening は、実際の型付き写像を記録する。
- canonicalization と intrusion は、元の identity を消さず、現在の端点の解決方法を更新する。

主張。 任意の処理段階で、すべての生成済みレコードから元の source rule まで有限の導出を辿れる。

証明。 レコード生成の回数について帰納法を使う。初期 source レコードは結論を満たす。新レコードは既存レコードを親に持ち、その親の導出に今回の一操作を付加できる。生成済みレコードを参照するため、この導出は生成順に良基礎である。代表変更は元のレコードを破壊しないので、この性質を保つ。□

これは provenance の完全性の証明であって、各 rule の意味保存の証明ではない。各 rule の保存を別途証明すれば、同じ帰納法で意味保存を合成できる。物理 ID や slot から後になって source origin を逆推定する必要が減る。

6.8 この四つの命題が閉じる範囲

以上から得られるのは、**完全な依存集合を持つ有限な制約 graph を、外側の条件を固定して捕捉し、整合した名前変更で使用する操作の保存**である。具体型 adaptation の推移性、有限個の型候補、Oracle の健全性は仮定していない。

残る実装への義務は、実際の `dep` の網羅、局所座標の一般化条件、anchor の自由な局所名の非参照、既存演算の同変性、全生成経路の登録、intrusion 後の索引、公開後の更新規律である。これらは具体的な所有者を持つ。R71 と R72 などの source-to-consumer 問題を、この契約に対する実装対応としてまとめる。

7 Effect と level を同じ証明に載せる

7.1 Effect を family 名の集合だけで減算しない

同じ effect family でも、どの呼出し・使用・handler に由来するかによって、扱ってよい contribution は異なる。したがって、一般の handler を単純な family 集合の差だけで説明しない。

説明用の有限モデルとして、effect contribution の集合を T とし、各 $t \in T$ に family、元の occurrence、use、scope に関する情報を持たせる。handler の許可を h、その handler が実際に扱うことを許される関係を $match(h,t)$ とする。

$$res_h(T)=\{t \in T : \neg\, match(h,t)\}.$$

これは既存の PUSH/POP と context 代数を置き換える提案ではない。証明しなければならない性質を、最小のモデルで明確にするための定義である。再開・順序・複数回の同じ operation を観測する場合は、trace や continuation を含む実際の意味論が別に必要である。

7.2 補題 5 残余と同時の名前変更

handler と contribution の名前変更が同じ写像 μ によって行われ、

$$match(h, t) \Leftrightarrow match(\mu h, \mu t)$$

であるとする。このとき

$$\mu(res_h(T)) = res_{\mu h}(\mu T).$$

証明。 左辺の元は、ある $t \in T$ で match しないものの像 μt である。同変性により、これは μT に属し μh と match しないことと同値である。集合の元ごとに両辺の同値が得られる。□

さらに、handler h が扱うと認められた contribution 以外は定義上消えない。freshening が handler の情報だけ、あるいは effect の情報だけを写すと、この命題の仮定が壊れる。

実装への接続。 `FormalPermission` の存在から直ちに match を成立させてはならない。同じ actual provider、同じ arrival、同じ source occurrence、同じ use と context の証拠を接続して、実際の許可関係を導出する。未知の coverage は不許可の証明でも許可の証明でもない。

7.3 既存の context 代数を再利用する

Yulang2 の `compose_for_replay` では、left と right の合成順序が異なる。right は入れ子になった wrapper として扱われるため、合成が逆順になる。現行の `evaluate_context` にも同じ順序が存在する。これを共通の context 演算として扱い、denotation を保存する補題を一度証明する。[R20]

ただし、現在の `LocalWeight` は left word と right pop の成分を `[(); 0]` として持ち、`materialize_unit_push` も live consumer のない実験的な操作と明記される。`evaluate_context` に代数が実装されていることは、一般の PUSH/POP が source から実行まで完成した証拠にはならない。代数の証明、source が作る context の制約、solver 内での停止性、runtime での handler 選択はそれぞれ接続が必要である。[R20][R21]

7.4 Existential の非脱出を導入規則から示す

rigid identity の scope を s、型変数 a が参照してよい rigid identity の集合を $Allowed(a)$ と書く。level に基づく導入規則で、この集合を決める。意味に必要な bound と payload について

$$RigidFV(bounds(a)) \subseteq Allowed(a)$$

を保つことを不変条件にする。ここで bounds の自由変数は、物理的に直接保存された一段の構文だけでなく、依存先の readback と必要な payload を含む意味的な自由 support とする。`a` から `b`、`b` から rigid identity への間接的な脱出も対象である。公開 root にも外側の環境が許す scope を割り当て、その root 自体と effect・証拠の support を検査する。これは新しい独立した scope 判定器を必ず作るという意味ではない。全派生比較に戻る既存 level guard と変換が保証すべき内容の数学的表現である。

証明は制約導入、伝播、extrusion、intrusion、一般化、freshening の各操作について行う。導入 level l に対する $\leq l$ の禁止比較は、alias と replay を含めて毎回同じ guard へ戻す。柔軟変数をコピーしても rigid identity 自体の level を下げない。同じ level の異なる scope を、level が等しいという理由だけで同一視しない。

existential を隠したまま運ぶ package や、その型だけを公開する操作を、一律に「内部 identity を含むから拒否」としてはならない。非脱出の対象は、binder の外へ自由に露出する identity である。この free と bound の区別を、source rule と全変換で保つ。

この不変条件が保たれるなら、公開される型・effect・必要な証拠に、許可されない自由な rigid identity が出現しないことが帰納的に従う。実際の existential の導入・除去規則を含む進行性は、第 8 章の型安全性の証明で別途扱う。

8 全体の健全性と主型性の証明計画

8.1 まず三種類の意味を定める

宣言的な source typing。 ソース e の型と effect の判断を

$$\Gamma \vdash e : A \,!\, \varepsilon$$

と書く。どのプログラムを受理したいかを定める規則であり、現在の solver が成功すること自体をこの判断の定義にしない。

制約生成と充足。 source から graph、root、元の Call との対応、必要な実行証拠の構造を作る。第 6 章の $\rho \vDash C$ は、独立して定義した宣言的な意味に対する充足である。

実行意味論。 計算状態に environment、store、handler と continuation の状態を含め、value へ進むか、許可された operation request を出すか、次の計算へ進むかを定める。runtime の具体的な implementation と対応させる対象となる。

この三つの間に定理を立てる。これにより、solver の保存性を証明しただけで runtime safety まで得たように扱うことを避けられる。

8.2 各操作で必要な保存方向

通常の伝播や分解については、同じ変数集合なら解の意味を保つことを示す。内部変数を追加する更新や表現を同一化する更新は、適切なモデル対応と、元の source interface への観測の保存を示す。単なる fresh な補助変数の導入であれば、元の変数集合 V に対して、例えば

$$\pi_V \, Sol(C') = Sol(C)$$

という形になる。ただし source の新しい制約そのものを追加する場合は、その新制約との連言を右辺に置く。ソースが要求した制限と、solver が勝手に解を減らしたことを区別する。内部の親子コピーを同一化する intrusion に、元の内部割当をすべて残す式を無条件に課さない。

部分的な主張にも方向を明記する。右から左への延長がなければ正常な解を失う可能性があり、左から右への制限がなければ不正な解を追加する可能性がある。型の健全性と自然な推論の完全性では必要な方向が異なる。

等式による商の補題。 occurrence が要求する親子等式の有限集合を E とし、生成された同値関係の商写像を q とする。対象は許可された変数の sort に限り、scope を保つ合法な置換とする。制約とその意味的 payload は同じ置換に従い、元の source provenance は別に保持する。このとき

$$Sol(q(C)) \cong \{\rho \in Sol(C) : \rho \vDash E\}.$$

証明。 商上の割当 v は $v \circ q$ によって元へ引き戻せる。同じ同値類にある変数は同じ値を持ち、合法な置換による term の解釈の一致から、各制約を満たす。一方、E を満たす割当は各同値類で一定なので、商へ一意に降りる。二つの構成は互いに逆である。□

これは既存の quotient solution correspondence の代数的な核を再利用したものである。[R26] 得られるのは「等式を課した元の解」との一致であり、元のすべての内部解が残るという結論ではない。必要な source typing が同じ型・effect・provider・証拠の観測を保って $C \wedge E$ に実現できることは、Value-SCC と occurrence の側で別途示す。

保存すべき観測の集合は、元の承認済み契約から固定する。都合の悪い all-view や将来の caller context を観測から外して、新しい定理の完了としない。permission や raw provenance の identity まで、型変数の商を理由に潰してよいわけではない。

8.3 純粋な小核の最初の縦断証明

literal、Name、Lambda、Apply、let、rec、if の source rule を同じ graph model 上で書く。最初の例として、純粋な式 $\lambda f.f\,1$ を考える。fresh な α, β に対し、f は α、呼出し結果は β であり、制約は

$$\alpha \preccurlyeq Int \to \beta$$

となる。root は $\alpha \to \beta$ である。通常の Simple-sub の純粋な矢印部分について、この制約の解から application と lambda の宣言的型付けを構成できる。極性に従って一般化・簡約した代表形は $(Int \to \beta) \to \beta$ となる。

この例の証明は、実際に呼び出される f を事前に決定することを要求しない。使用から制約を生成して、後から下界が入れば既存の上界へ replay する。`if true { 1 } else { "str" }` でも両枝の制約を蓄積し、結果には `int | str` が現れる。条件の評価や定数畳込みを型推論の前提にしない。[R12][R15]

この小核を完成品の代わりにしない。完成すれば、同じ生成健全性定理の新しい場合として effect、Tuple、method、existential、record を追加する。初期断片は証明の順序であり、最終的な対象言語の縮小ではない。

8.4 必須の補題と所有箇所

補題	示すこと	主な所有箇所
生成健全性	graph の解から同じ source の宣言的型付けを作れる	HIR と `candidate_source`
生成一般性	宣言的な型付けを graph の解へ持ち上げられる	同じ source rules
伝播保存	各 worklist rule が余分な解を作らず、必要な解を失わない	session と context
extrusion 保存	極性付きコピーと境界への射影が一致する	`candidate_extrusion`
intrusion 保存	実際の occurrence と Value-SCC 条件で同一化を正当化できる	value graph と intrusion
Capture と FreshUse	第 6 章の依存閉包と名前変更の仮定を実装が満たす	`candidate_scheme`
公開射影	graph の観測可能な型と出力 scheme の意味が一致する	export と簡約
実行対応	source typing の Call・handler 証拠を実際の runtime が消費する	playground runtime

生成健全性と生成一般性は、ソース構文に関する帰納法で示す。Call の場合は、AV、AE、RE、RV と handler の証拠が一つの実際の呼出しを説明することを使う。let の場合は、一般化可能性の side condition と第 6 章を使う。rec の場合は、SCC の再帰参照とコピーの対応を使う。

8.5 合成定理としての推論健全性

当初定めた source と合法な使用の対象範囲について、次の条件が成立するとする。断片で先に結果を得ても、未対応構文や承認済みの将来の caller context を除外して親定理の完了と数えない。

1. source の各生成規則について、生成健全性が証明されている。
2. solver の各更新、extrusion、intrusion が、元の変数への射影を含めた必要な解保存を満たす。

3. 成功して公開する状態について、未処理の型エラーや必要な制約の脱落がない。公開 scheme の任意の許容 instance は、同じ最終 graph のモデルとして実現されるか、その instance の typing derivation を直接検証できる。
4. 一般化と freshening は第 6 章の仮定を満たし、一般化の side condition も成立する。
5. 公開射影は graph の宣言的な型観測を保存する。

このとき、成功して返された scheme のすべての許容 instance は、同じ source の宣言的型付けによって正当化される。

証明。 返された scheme の任意の許容 instance を固定する。条件 3 でその instance の typing derivation を直接検証した場合は、結論を得る。モデルを用いる場合は、条件 2 と 4 を実行された有限の変換列に沿って合成し、同じ観測を保つ元の source 制約の解へ戻す。条件 1 を構文に関する帰納法で適用し、ソース全体の型付けを得る。条件 5 により、その型付けが固定した公開 instance を正当化する。instance は任意だったので結論を得る。□

条件 3 は「worklist が空だから充足可能」と置き換えられない。一つの解が存在するだけでも、返した scheme 全体の健全性には足りない。失敗検出、循環した制約の解釈、guarded な再帰型、公開 instance の実現を所有者で示す必要がある。現行の unresolved な candidate solve には、この合成定理をまだ適用できない。

8.6 型安全性には preservation と progress が必要

実行状態の型付けを定義し、次の二命題を示す。

Preservation。 型付けされた状態が一ステップ進むと、store typing などの必要な拡張のもとで、次の状態も同じ外部契約を満たす。handler の起動、resume、fresh な実行 identity、method adaptation、existential package を含めて証明する。

Progress。 閉じた型付け済み状態は、value であるか、外側の effect 契約が許す operation request であるか、次のステップへ進める。effect が空で必要な runtime handler が整っている場合には、不正な適用や禁止された unhandled operation で stuck しない。

実 runtime との対応。 宣言的な状態 s と evaluator の状態 m の対応関係を定める。同じ binding、actual provider、引数と結果、store、handler activation、残りの continuation を対応させ、fresh な実行 identity は一貫した名前対応のもとで比較する。

初期 elaboration が対応する状態を作ること、runtime の各 step を宣言的な有限 step で模擬できること、operation と戻り値の観測が一致することを、Call、request、handler 選択、return、resume などの場合ごとに示す。さらに、対応する runtime が型の不一致で早く停止しないことを示す。step の模擬だけでは、step 自体を出さずに stuck する実装を排除できないためである。

この入口は、推論と elaboration が作った同じ型付き結果を runtime が消費することである。通常の `RuntimeEvidence::default()` と限定された operation v1 の receipt 経路の差を、未完成の対応として埋める。テスト時だけ別に正しい証拠を渡しても、通常 source 経路の定理は証明されない。

型安全性。 推論健全性により得た source typing と、source から runtime への対応を使って初期状態を型付けする。preservation を有限ステップ数について帰納的に適用し、progress と合わせる。これにより、受理したプログラムが型に起因する不正状態へ到達しないことが従う。発散するプログラムが存在しないという主張ではない。

8.7 主型性は解の存在より強い

主型性では、返した型が正しいだけでなく、宣言的に許されるすべての型付けがその instance として表されることを示す。固定した環境と許可された scope に相対化し、同じ outer anchor を保持する必要がある。

各目標の宣言的型付けについて、binder、outer anchor、共有変数、必要な証拠を保つ一つの合法な構文的 instance 写像を構成する。この写像を、個々の ground 割当や未来の実行履歴を見た後に選び直してはならない。各解を別々の対応で覆えるという主張だけでは、構文上の主型性とは限らない。

joint な関係を各 root の独立な観測へ置き換えることにも注意が要る。例えば (int, int) と $(bool, bool)$ だけを許す関係を、二つの root がそれぞれ `int | bool` とだけ表示されるものへ置き換えると、元にはない $(int, bool)$ を許してしまう。内部の相関を保持することと、その意味を保つ公開型へ射影することは別の証明義務である。

必要な追加条件は、生成一般性、solver の解延長の方向、intrusion の一般性、一般化してよい変数の完全な選別、公開射影の両方向の意味保存である。第 6 章は、そのうち Capture と FreshUse で解を失わない部分を担当する。

method に未解決の義務が残る場合、その義務を失った通常の monotype だけを主型として公開しない。既存の意味論で許される形の残留義務か、正当化された解決証拠を保つ。競合する method 候補から恣意的に一つを選ぶ操作には、一般性の証明は付かない。

公開型の短い表示、内部 graph の解集合、all-view の完全特徴付けを区別する。強い特徴付けの証明が未完であっても、必要な自然な推論の一般性を捨ててよいわけではない。逆に、既存契約が必要としない強い特徴付けを自動的に全実装の停止条件にしない。[R11][R14]

8.8 停止性と順序独立性

有限な事実集合 U と、各規則の有限な局所処理を考える。各規則が冪集合 $P(U)$ 上の単調かつ増大する作用素であり、同じ事実を重複登録せず、新事実が現れたときだけ有限個の work を登録するとする。単なる追記専用データ構造という意味ではなく、前提が増えたときに以前の帰結を否定しないという意味で単調とする。さらに、再処理を要する規則は必要な前提の更新で必ず起動されるとする。

この場合、事実の真の追加回数は U の要素数を超えない。各追加が生む仕事が有限なので、全体も有限で停止する。公正にすべての規則を処理すれば、結果は初期事実を含む最小の閉包であり、worklist の処理順によらない。これは通常の有限な単調飽和の証明である。

Yulang で残る核心は、U の有限性を示すこと。 有限なソースであることだけでは、無制限の context word、fresh identity、コピー、再帰的な effect の状態が有限になるとは限らない。以下を具体的に示す必要がある。

- コピーの cache key が、元の変数、極性、境界、使用の有限な情報で決まり、同じ状態を再生成し続けない。
- SCC の merge と canonicalization が、既存の処理済み条件を失わせない。
- source が生成する context の正規形または有限商が、実際の Call・handler 観測を保存する。

- 処理済みの pair が後から bounds を得たとき、必要な再起動があり、同じ未変化状態を無限に処理しない。

context を単なるカウンタへ潰す、固定長で切り捨てる、任意の正規化を仮定する、といった変更では証明にならない。有限商の実現が困難なら、具体的な source grammar から有限性を導くか、独立した良基礎の尺度を作る。資源上限による安全な中断は実装の保護策であり、数学的な停止性・完全性の代用ではない。

9 実装順序と各段階の完了条件

順序を変える必要がある場合も、各段階は「対応する証明」「実装」「元のソースでの観測」を一組として扱う。全言語の一般定理が完成するまで、正当化された独立部分の実装を止める必要はない。一方、核心の前提が未解決のまま大きな機構を積み増さない。

段階 0 Oracle と現在地を固定する

既存の differential driver と paired source cases を再利用し、Oracle SHA、candidate SHA、toolchain、flags、source 方言の変換、binary hash を manifest に持つ runner を作る。古い `/tmp` の実行ファイルと固定した過去の native hash を、現在の比較条件にしない。[R23]

最初の出力は、各 case の accept、type error、unsupported、resource limit、timeout を分けた結果とする。型・effect・runtime trace が未取得なら、その項目だけ未取得にする。

完了条件。 同じ固定ソースを同じ条件で再実行でき、誤拒否と未実装を区別できる。型名の alpha-renaming による差を、意味の差と誤認しない。

段階 1 意味的な incidence と Capture を直す

対象は `candidate_context.rs`、`candidate_scheme.rs`、`candidate_extrusion.rs`、必要な session の admission 部分である。

1. 既存の bound identity と typed cause を正本に定め、意味が参照する全 payload の依存先を列挙する。

2. 片側所有を維持しながら、両端と必要な payload から辿れる索引を持つ。
3. source completion の観測記録を Capture の採否条件から外す。必要な semantic 登録の失敗時は、その private result を公開しない。
4. 第 6 章の閉包を `Capture` の完了条件にし、outer anchor の固定と任意長の incoming chain を扱う。
5. 現在の map を使って全制約と payload を復元し、元の occurrence との対応を保つ。

完了条件。 特定の slot、shape の一段の探索、任意に省略される receipt がなくても、必要な制約が shared または captured のどちらかで保存される。equal-age bound、older anchor、canonical merge、再帰 SCC、独立二使用を同じ命題で説明できる。

公開後の late bound も検査する。既存の generic graph を更新する経路があるなら、公開境界の閉鎖条件、または必要な instance への invalidation/replay を証明する。固定した Capture の保存と、後続更新の保存を混同しない。

ここで閉じるもの。 Capture と FreshUse の境界保存。全言語の型安全性や、任意の context の停止性まで閉じたことにはしない。

段階 2 function と effect を端から端まで接続する

source の literal、Name、Lambda、Apply、let、rec、if について、生成規則と実装の対応表を完成させる。Call の四ポートと permission を一つの導出にし、通常実行へ同じ型付き結果を渡す。限定された operation/receiving 実験の成立範囲を、一般経路と明確に対応させる。

R252 の `[io,'e]` と callback 残余を、この段階の代表例にする。handler の減算、higher-order forwarding、returned closure、二つの fresh use を同じ context 代数で通す。

完了条件。 受理の一致だけでなく、同じ source/use の residual と実行時の handler 対応が成立する。生成健全性、生成一般性、既存 hygiene 補題の適用条件が、source constructor ごとに埋まる。

段階 3 Tuple と pattern を共通 traversal へ載せる

順序付き product constructor を term に追加し、constraint generation、Capture、extrusion、freshening、公開型、評価を同時に接続する。構造の子を列挙する共通インターフェースが有効だが、variance、binder、評価順序の違いは明示する。

完了条件。 source の Tuple の順序と要素数が保たれ、各要素の effect が正しい順に実行される。型だけ Tuple を認めて Capture が子を落とす、といった経路がない。R149 系の元のサンプルについて、Tuple 未対応を source rewrite で回避しない。

段階 4 method と optional record を遅延義務として統合する

Yulang2 の lookup、role、適合の正常例を固定する。型が未確定なら同じ session に義務を残し、関係する bounds が増えたとき再起動する。選択可能になる前に具体的な method を決めない。

選択規則が後から現れる情報で変わり得るなら、未確定の選択を早期に公開せず、規則の安定条件を証明する。候補の確定が単調な作用素として表せない場合は、第 8.8 節の飽和が完了した後の別段階として扱い、その確定が以後の合法な更新でも保たれることを示す。複数の候補型付けを実行して成功したものを選ぶ solver にしない。optional record は局所 adaptation と witness を生成し、具体型関係の推移閉包を導入しない。

完了条件。 どの更新が義務を起こし、何をもって選択が確定し、その証拠を Call と runtime がどう使うかが一つの規則で説明できる。正常な Oracle 例を保ち、曖昧さや未実装を恣意的な型エラーに変えない。

段階 5 existential と一般 handler を接続する

fresh level l の rigid 導入と $\leq l$ への比較・同一化禁止を実装し、alias と replay を含む全派生比較を同じ guard へ戻す。第 7 章の非脱出不変条件を全変換へ接続する。型付きの package と binder の内側に隠された identity を、自由な脱出と区別する。

generic operation arm、continuation と resume の型付けを同じ scope 契約へ追加する。現在の monomorphic な限定を外すときも、単に guard を削除するのではなく、対応する生成・実行の証明を追加する。

完了条件。 rigid identity の自由な脱出がなく、正当な existential の利用は受理される。専用の quantified solver や後付けの scope-exit 検査を、証明の主経路へ置かない。

段階 6 公開 scheme と旧経路の撤去

内部 graph の意味と公開 scheme の意味を両方向に結び、主型性の合成を閉じる。実行に必要な private identity を消すときは、その前に runtime evidence へ正しく受け渡す。

旧 F5 依存の consumer を列挙し、置換済みのものから一つずつ撤去する。LocalSource と旧 HIR の二重経路も、同じ規則へ収束させる。旧ノートは履歴として残し、新構成で不要となった proof edge は、置換先と理由を記録して引退させる。証明済みと偽って閉じない。

完了条件。 外部向けの推論・公開・研究 evaluator の入り口が同じ意味を持ち、旧経路への fallback が自然な受理を支えている箇所が残っていない。VM/native backend の完成は、研究用 evaluator の完成と別の実装範囲として扱う。

10 検証する例と進捗の測り方

10.1 統合回帰の最小構成

群	固定する入力	受理以外に確かめる観測
基本関数	identity と高階適用	型の共有と制約生成の対応
多相使用	同じ identity の int と str での使用	local の独立性と outer の共有
条件分岐	定数条件でも異なる型の両枝	union と両枝の制約
再帰	相互再帰と外への返却	実参照 SCC と live root
R252	negative formal に state callback	called 時の残余、positive control

群	固定する入力	受理以外に確かめる観測
handler	別 family と同 family の別由来	許可された contribution だけの除去
forwarding	複数段の relay と generalized use	必要な全 chain の輸送
returned closure	handler 境界を越える callback	実 provider と invocation の対応
method	後から bounds が増える受け手	再起動と選択証拠の安定性
existential	正当な package と不正な自由脱出	rigid identity と level の保持
optional record	欠落・存在する field の利用	局所適合と実際の field 安全性
元の Yulang2 samples	Types と Console 等	import から runtime までの各段階

各群には、正常例と、それに近い不正例を入れる。拒否例だけを増やして通過率を上げない。R252 のように以前の拒否前提が誤りだった場合は、受理を保持したうえで残余の検査を追加する。

10.2 比較結果の分類

分類	意味	次に行うこと
一致	現在取得できる観測が一致	一般証明の代用にはしない
実装不足	source または consumer がない	実装の所有箇所へ戻す
candidate 回帰	承認済みの挙動から逸脱	元の規則と消費点を同時に直す
Oracle の立証済み不具合	具体的な反例が揃う	修正規則と近傍正常例を固定
仕様訂正	意図する意味が明示された	訂正された意味で比較を更新
未決着	同じ source/use の対応が不足	不足している一つの対応を調べる

公開主型の一般同値が決定できない段階では、alpha 同型、固定した ground instance、具体的な使用例の比較を行う。その結果を一般的な主型性の証明と呼ばない。

10.3 正確性に必要なテストへ絞る

Capture の索引を変更するなら、制約の生成方向と物理 owner が反転する例、incoming chain、canonicalization 後の参照、shared anchor、二つの使用を重点的に検査する。receipt の保存失敗を意味的に扱う必要がなくなったなら、診断情報の有無で観測が変わらないことを確認する。

新しい source constructor を追加するなら、その一個を含む縦断例と、全変換で子を訪れることの検査を行う。記録文書だけを編集した後に、重い全 suite を繰り返さない。テストの追加自体を進捗指標にしない。

10.4 計算量を測る単位

最低限、採用した意味的制約の数、incidence の数、work item の処理回数、実際に作ったコピー数、context の異なる状態数、Capture が訪れた範囲を記録する。両端索引は既存の辺を参照する疎な索引とし、二重の意味的正本にしない。

Capture は第 6 章の到達した incidence に比例する実装を目標にする。bounds の交差 replay は、実際に必要な組の数に費用を課す。context の状態が増える場合は、その生成理由を測る。根拠なしに全 solver が線形時間であると約束しない。

性能を悪化させる全面的な graph 複製、毎回の全 source 再走査、無関係な owner の探索は避ける。最適化する場合は、同じ root 観測を保つことと、削減した仕事量を示す。

11 文書と作業単位を一つの証明へ集約する

11.1 正本に必要な内容

今後の統合文書は、次の順に固定する。

1. 対象 source と意図する型判断。
2. 型・effect・scope・Value 参照の共通モデル。
3. 各 source rule と、それが作る制約。
4. solver の更新規則と不変条件。

5. extrusion、intrusion、Capture、FreshUse の保存。
6. 公開 scheme と runtime の対応。
7. 健全性、主型性、停止性の合成定理。
8. 各定理の実装所有者と回帰例。

既存の証明ノートは、この正本の補題の証明または補足として参照する。日付付きノートを読まなければ現行の定義が分からない状態にしない。歴史的な代替構成を、現行の証明の前提として残さない。

11.2 既存課題との接続

既存の課題	統合文書での責任	完了の証拠
R71 と R72	source occurrence から実際の bound と consumer までの保存	全生成経路の導出と同じ map の適用
R239 から R246 の Capture 議論	境界を固定した依存閉包と解の分解	第 6 章の仮定を実装が満たす証明
R252	negative row と permission の意味	受理、消費、残余、fresh use の一組
SOURCE_ADEQUACY	source typing と graph の双方向対応	構文帰納法の全場合
GENERALIZE と PROJECTION	局所 graph の保持と公開観測	境界保存と射影の両方向
SOUND と PRINCIPAL	上記を元の対象範囲へ合成	未証明の前提が残らない導出
停止性の課題	source が作る状態の有限性または良基礎性	context とコピーを含む尺度

R71 や R245 が未解決であること自体を、source-level の反例が得られたという意味に変えない。R246 は、現在の fixture が全 same-use constraint の inventory を assert していないことを指摘している。印字された一部の観測だけで、完全な反例や全称的な保存を結論しない。[R24][R25]

11.3 一つの作業に必ず結び付ける項目

各作業は、同じ文書内の一項目として、元の命題、触る source constructor、生成する制約、全 consumer、保存不変条件、実装ファイル、確認する正常例と不正例、残る前提を持つ。条件付き補題ができたら、その残る前提を具体的な所有者へ渡す。

「新しいノートを作った」「レビューが通った」「局所テストが増えた」という事実だけで、親定理を閉じない。一方、すでに成立している hygiene の補題を、表現が変わったという理由だけで最初から証明し直さない。写像と仮定の対応を示して再利用する。[R12][R14]

11.4 直近の割当案

最初の実装担当は、段階 1 の semantic incidence と Capture に集中する。証明担当は、第 6 章の依存集合と実際の payload の対応、outer anchor の扱い、全 consumer への適用を担当する。別の確認担当は、現行の生成経路を列挙し、保存されない辺や別 use の情報の混入を探す。

並行して、段階 0 の Oracle runner と、既に仕様が定まった Tuple の constructor 対応表を準備できる。型の核心の証明が必要な新規機構を、無関係なサンプル専用の修正で先に固定しない。

この作業の完了報告は、「Capture の必須情報が観測依存から外れた」「任意長の局所依存閉包が保存される」「同じ use の map で全 constraint が戻る」という三つの具体的な結論を中心にする。そこから function/effect core の生成健全性へ進む。

12 確認結果と参照資料

12.1 今回確認した範囲

固定 commit の候補推論、Capture、extrusion、intrusion、effect、source lowering、playground runtime、証明課題と関連ノートを読んだ。別の Yulang2 checkout で参照操作の実装と版の差分を確認し、Simple-sub の原論文も確認した。

この環境には Rust の toolchain がなかったため、コンパイラの新しいビルドや回帰テストは実行していない。テストソースの読解は、「そのテストを実行すれば何を確認するか」の証拠であり、実行成功の報告ではない。リポジトリのコード、ブランチ、既存 task 状態は変更していない。

第 6 章と第 7 章の補題は、ここに明記した抽象契約に対する証明である。実装への適用に必要な未充足の仮定は、各節と第 8 章に残した。現在の Yulang 全体を不健全と断定する新しい source-level 反例も、全体の健全性完成も主張しない。

12.2 Simple-sub 原典の利用範囲

Parreaux の原論文は、上下界の伝播、極性付き extrusion、一般化と freshening の実装原理、および健全性・完全性の証明方針の一次資料である。形式化の節は non-let-polymorphic な範囲に限定され、完全な証明は省略されている。したがって、Yulang の effects、Value-SCC intrusion、一般の let、公開 scheme、runtime safety が引用だけで証明されるわけではない。[R15][P01]

通常の部分型推論の既知の結果は再利用し、追加した意味に必要な差分だけを証明する。その差分と元の命題の前提を、一つの文書で追えるようにすることが本書の方針である。

12.3 ソースへの固定リンク

以下の R01 から R29 は、この文書の事実確認と実装の引継ぎに使う固定位置である。単一ファイルに複数の機能がある場合は、関係する記号と行を添えた。

- **R01 研究ブランチの基準。** commit 5596b159。
- **R02 候補 API と未解決前提。** shadow_apply.rs。11–79 は未解決条件、928–969 は候補 solve と preflight。
- **R03 共通 session と候補の終了条件。** yu-solver/src/lib.rs。4043–4046 は adjacency 所有、10425–10465 は実行と終了、11810–11819 は worklist。
- **R04 Capture と reverse index。** candidate_scheme.rs。821–928、1289–1405 は capture frontier と reverse owner。
- **R05 source completion の省略。** candidate_context.rs 2150–2161。

- **R06 source rule と未対応箇所。** candidate_source.rs。772–803 は opening と移行経路、1191–1208 は Catch、1509–1513 は Tuple。
- **R07 playground の推論と実行。** playground lib.rs。171–183 は通常実行、219–236 は operation v1、285–325 は source inference。
- **R08 Role inventory。** shadow.rs 917–925。
- **R09 annotation carrier と runtime。** source_annotation.rs 79–95 の型 constructor、runtime/program.rs 567–571 の Tuple。
- **R10 R252 の意味と検査範囲。** 修正ノート、candidate_effect.rs 231–234 と 1397–1427、CLI 回帰 30–79。
- **R11 統合証明の依存関係。** successor-proof-obligations.md。SOUND、PRINCIPAL、SOURCE_ADEQUACY、ALL_VIEW、PROJECTION の項目。
- **R12 利用者が承認した上位原則。** top-level principles。
- **R13 Value graph と intrusion。** candidate_value_graph.rs と candidate_intrusion.rs。
- **R14 設計の優先順位と level 規律。** design-authority.md、redesign charter 第 22 節と第 23 節。charter 全体を承認済みと読み替えず、個別に記録された利用者の決定を使う。
- **R15 原典の証明範囲の監査。** Parreaux proof import。
- **R16 Oracle の bounds と extrusion。** bounds.rs。630–735、815–900、4523–4544、4701–4727。
- **R17 Oracle の関数比較。** propagate.rs 212–270。
- **R18 Oracle の source application。** tail.rs 535–628。
- **R19 Oracle と候補の instantiation。** instantiate.rs 620–785、candidate_scheme.rs の `prepare_candidate_fresh_use` と restore。
- **R20 context の合成。** Oracle ConstraintWeights、candidate_context.rs の `LocalWeight`、`materialize_unit_push`、`evaluate_context`。
- **R21 Oracle の handler 残余。** row_effect.rs 88–233。

- **R22 Oracle の未決着と過去の反例の範囲。** direct provider 監査、旧推論文書の監査。
- **R23 既存の Oracle 実験 driver。** research_playground_hygiene_oracle.py。
- **R24 Capture の条件付き障害。** R244、R245。
- **R25 source qualification の未充足条件。** R246。
- **R26 等式の商と残る source bridge。** quotient solution correspondence 25–58 と 128–180。
- **R27 既存の統合証明方針。** successor production proof architecture。
- **R28 非推移的な具体型比較。** finite bound replay closure 第 4 節。
- **R29 Oracle の注釈。** annotation/constraints.rs 299–470。
- **P01 原論文。** Lionel Parreaux, *The Simple Essence of Algebraic Subtyping*, PACMPL 4, ICFP, Article 124, 2020. DOI、著者の公開ページ、固定した参照実装。